Intermediate Statistics using R - Part 1

Contents

1. Introduction to R and RStudio . . . 2
    - 1.1. R . . . 2
    - 1.2. RStudio screen . . . 2
    - 1.3. Interacting with R . . . 2
    - 1.4. Assignment Operator . . . 3
    - 1.5. R help . . . 4
    - 1.6. Commenting . . . 4
2. Data Types and Structures . . . 4
    - 2.1. Data Types . . . 4
    - 2.2. Data Structures . . . 5
    - 2.3. Vector . . . 5
    - 2.4. Missing Data . . . 6
    - 2.5. List . . . 7
    - 2.6. Data Frame . . . 8
3. Referencing Data . . . 8
    - 3.1. Interpreting Rows as Headers . . . 9
    - 3.2. Referencing by Index . . . 10
    - 3.3. Referencing by Name . . . 11
    - 4.4. Referencing by Logical Vector . . . 12
    - 3.5. Combining Referencing and Assignment . . . 13
4. Understanding Factors . . . 14
    - 4.1. `factor()` Function . . . 14
5. Packages & Reading and Writing Data Files . . . 14
    - 5.1. Packages in R . . . 15
    - 5.2. Read and Write Data Files . . . 16
    - 5.3. `tydyverse` Package . . . 17
6. Ploting in R (optional) . . . 19
    - 6.1. Basic Ploting . . . 19
    - 6.2. Scatter Plot . . . 21

6.3. Bar Plot 22
6.4. Pie Plot 24
6.5. 3D Pie Plot 26
6.6. Histogram 27
6.7. Box Plot 28

1. Introduction to R and RStudio

Objectives

- Get familiar with the R and RStudio interfaces.
- Understand the difference between a script file and the console.
- Learn and apply useful functions and shortcuts.

1.1. R

- A language designed for statistical computing and data science
- Boasts millions of users worldwide
- Entirely free and open source!
- Offers best-in-class data visualization
- Thousands of R packages available (extensions to R)
- Widely used in statistics, data mining, bioinformatics, etc.
- Vibrant and helpful community
  - Explore R bloggers for more insight

1.2. RStudio screen

- Top left panel, Script Editor, displays the code of the currently open file.
- Top right panel shows a list of workspace variables along with their corresponding values.
- Bottom left panel, Console, executes code and displays the output/results.
- Bottom right panel provides access to the help menu, a list of packages, plots, etc.

1.3. Interacting with R

Interaction with R primarily happens in two ways – (1) through the Console or (2) Script Editor.

The **Console**, found in the bottom left panel of RStudio, is where R is waiting for your commands and displays the resulting outputs of executing the commands. Commands can be directly typed into the Console, but be aware that they are not retained once the session is closed.

For more permanent command tracking, it is advised to enter commands in the **Script Editor** and save the script. This creates a comprehensive record of your actions, simplifying the process of demonstrating your work to others or reproducing your steps in the future if necessary.

While it is possible to copy-paste into the Console, the Script Editor provides a more streamlined option. You can 'send' the current line or the selected text to the Console using the `Ctrl + Enter` shortcut (`Command + Enter` for macOS) .

To execute commands, type the text into the Script Editor (located in the top left panel of RStudio), hit `Ctrl + Enter` for each line, and observe the resulting actions in the Console window.

During your analysis, there may be instances when you wish to inspect the contents of a variable or the structure of an object, 'without' necessarily documenting it in your script. To facilitate this, RStudio offers `Ctrl + 1` and `Ctrl + 2` shortcuts for easy navigation between the Script and Console windows. Let's give it a try.

When R is ready to accept commands, the Console displays a `>` prompt. Upon receiving a command, R attempts to carry out the command, and once finished, it exhibits the results and returns with a fresh `>` prompt to await further instructions.

If R is anticipating additional input because your command isn't yet complete, the Console will present a `+` prompt. This indicates that you haven't concluded your command, possibly due to an unclosed parenthesis or quotation. Should this occur in RStudio, simply click within the Console window and press `Esc` – this action will assist in resolving the issue.

1.4. Assignment Operator

`<-` serves as the **assignment** operator. It assigns the values on its right to the objects on its left. For instance, following the command `x <- 2`, the value of x is now 2. This operation can be interpreted as "2 goes into x". A convenient way to type an assignment operator is to use `Alt -` (`Option -` for macOS). Variable and function names are all case sensitive, meaning R assume upper case and lower case names are different.

Although `=` can also be used for assignments, it is not universally applicable. Thus, it is recommended to use `<-` for assignments. The `=` operator should be reserved for specifying the value of an **argument** within functions.

```
x <- 2 # lower case x
x
```

```
## [1] 2
```

```
X <- 10 # upper case X
X
```

```
## [1] 10
```

```
x
```

```
## [1] 2
```

```
y <- 4
y
```

```
## [1] 4
```

```
x * y
```

```
## [1] 8
```

```
xy <- x * y
xy
```

```
## [1] 8
```

1.5. R help

To learn how to use a function (e.g., to see the required parameters), type a question mark followed by the function name.

```
?sum
```

1.6. Commenting

The `#` sign is used to create **comments** within your R script. Generous commenting is highly recommended, as anything following a `#` is ignored by R during execution.

For swiftly commenting or uncomenting multiple lines, you can use the `Ctrl + Shift + c` shortcut (`Command + Shift + c` for macOS).

```
### R, please ignore me!
```

2. Data Types and Structures

Objectives

- Introduce learners to the various data types in R and illustrate their usage in data structures.
- Learn to create vectors of diverse types.
- Understand how to verify the type of a vector.
- Get knowledge about missing data and other special values.
- Get familiar with different data structures (lists, matrices, data frames).

To fully leverage the R language, it is vital to have a robust understanding of basic data types and data structures and how to operate on them.

Comprehending data structures is crucial, as these are the objects you will regularly manipulate in R. Handling object conversions can often be a major source of frustration for beginners.

2.1. Data Types

In R, 'everything' is treated as an **object**. R recognizes 6 basic data types:

- **Character**: "red", "green", "blue", 'red', 'green', 'blue'
  - Characters with single and double quotations work the same
- **Numeric**: 2, 15.5 (real or decimal numbers)
- **Integer**: 2L (the "L" indicates to R that this should be stored as an integer)
- **Logical**: TRUE, FALSE
  - Logical values can be combined with '&', '&&' (AND operator) or '|', '||' (OR operator) operator.
- **Complex**: 1+4i (complex numbers composed of real and imaginary parts)
- **Raw**: Raw data is usually stored in hexadecimal format, and it's used to represent byte-level data directly.

These data types can be combined to construct data structures such as atomic vectors. When we call a vector *atomic*, we mean that the vector contains data of a single data type only.

2.2. Data Structures

R encompasses several data structures, including:

- **Vector**
- **List**
- **Matrix**
- **Data frame**
- **Factors**
- **Array**

To explore an object's properties, R offers numerous functions, including:

- `class()` or `class()`: Shows the object's data type.
- `length()`: Shows the length of the object.
- `str()`: Shows the object's data structure and content.
- `attributes()`: Reveals any associated metadata.

2.3. Vector

A **vector** is a collection of elements. It can be created by directly stipulating its content, after which R will infer the suitable mode (data type) for storing the vector.

```
x <- "EPSY"
class(x)
```

```
## [1] "character"
```

```
length(x)
```

```
## [1] 1
```

```
y <- 5381
class(y)
```

```
## [1] "numeric"
```

```
length(y)
```

```
## [1] 1
```

The `c()` function, which stands for "combine", is frequently employed to create a vector. For example, the following code generates a numeric vector x.

```
x <- c(1.5, 3, 4.5)
str(x)
```

```
##  num [1:3] 1.5 3 4.5
```

If you want to specifically create integers, each element should be suffixed with "L" or you can 'coerce' the data into integer type using the `as.integer()` function.

```r
y <- c(1L, 2L, 3L)
str(y)
```

```
##  int [1:3] 1 2 3
```

```r
x
```

```
## [1] 1.5 3.0 4.5
```

```r
z <- as.integer(x)
z
```

```
## [1] 1 3 4
```

The `seq()` function is used to generate numeric vectors that contain a sequence of numbers.

```r
x <- seq(from = 1, to = 10, by = 2)
str(x)
```

```
##  num [1:5] 1 3 5 7 9
```

```r
y <- seq(from = 1, to = 2, by = 0.1)
str(y)
```

```
##  num [1:11] 1 1.1 1.2 1.3 1.4 1.5 1.6 1.7 1.8 1.9 ...
```

The `:` (colon) operator is used to create integer vectors that contain a sequence of integers.

```r
x <- 1:10
str(x)
```

```
##  int [1:10] 1 2 3 4 5 6 7 8 9 10
```

2.4. Missing Data

R accommodates missing data within vectors. These are denoted as `NA` (Not Available) and can be incorporated into all types of vectors.

```r
x <- c("a", NA, "c", "d")
y <- c(0.5, NA, 0.7)
z <- c(TRUE, FALSE, NA)
```

The `is.na()` function identifies the elements within a vector that are missing a value. The `anyNA()` function returns TRUE if the vector includes 'any' missing values.

```r
x
```

```
## [1] "a" NA  "c" "d"
```

```r
is.na(x)
```

```
## [1] FALSE  TRUE FALSE FALSE
```

```r
anyNA(x)
```

```
## [1] TRUE
```

2.5. List

In R, a **list** serves as a container. Unlike atomic vectors, the contents of a list are *not* restricted to a single mode and can hold any mix of data types. Due to their flexibility to include any type of R object – even lists within lists – they are often called 'generic' vectors. This property sets them apart from atomic vectors.

In simpler terms, a list is a special type of vector wherein each element can belong to a different type.

```r
x <- list("EPSY", 5381, 1L, TRUE, NA)
x
```

```
## [[1]]
## [1] "EPSY"
##
## [[2]]
## [1] 5381
##
## [[3]]
## [1] 1
##
## [[4]]
## [1] TRUE
##
## [[5]]
## [1] NA
```

The elements within a list are indexed by double square brackets `[[]]`. The contents of these elements can be retrieved by using double square brackets.

```r
x[[1]]
```

```
## [1] "EPSY"
```

The elements within a list can be named – this means lists can have the `name` attribute. If list elements are named, they can be referenced using the `$` notation.

```r
xlist <- list(a = "EPSY 5381", b = 1:10, data = head(mtcars))
xlist
```

```
## $a
## [1] "EPSY 5381"
##
```

```
## $b
##  [1]  1  2  3  4  5  6  7  8  9 10
##
## $data
##                    mpg cyl disp  hp drat    wt  qsec vs am gear carb
## Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
## Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
## Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
## Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
## Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
## Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

```r
names(xlist)
```

```
## [1] "a"    "b"    "data"
```

```r
xlist$b
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

Lists are incredibly useful 'inside' functions. Because R functions can only return a single object, lists enable you to 'bundle' various types of results into a single object that can be returned by a function.

2.6. Data Frame

A **data frame** is a very important data type in R. It's virtually the *de facto* data structure for most tabular data and is widely used for statistical analysis.

A data frame is a special type of list in which every element shares the same length – i.e. a data frame is a "rectangular" list.

We will delve deeper into this topic of data frames in the following section.

3. Referencing Data

Objectives

- Understand the three primary methods that R utilizes to reference data.
- Combine the data referencing methods with assignment operators to update subsets of data.

R is an efficient language for data manipulation. It primarily uses three methods to reference data within a data frame:

- Index (subsetting)
- Logical vector
- Name

Let's begin by (1) setting your working directory and (2) loading some sample data.

From the RStudio menu, **Session** » **Set working Directory** » **Choose Directory** » Navigate to the directory where you've saved the data files to work with » **Open**.

```r
getwd() # Check current working directory
```

```
## [1] "C:/Users/Jason Lee/OneDrive - Texas Tech University/files/ttu/education/classes/stats_soft/modul
```

```r
dat <- read.csv(file = "sample1.csv", header = TRUE)
#dat <- read.csv(file = file.choose(), header = TRUE) # select sample1.csv in File Dialog
#dat
```

3.1. Interpreting Rows as Headers

The first row of this .csv file represents column names. We incorporated the `header = TRUE` argument in the `read.csv` function so that R can interpret the file correctly.

Let's take a look at this data.

```r
class(dat)
```

```
## [1] "data.frame"
```

R has loaded the contents of the .csv file into an object named `dat`, which is a **data frame**.

The `str()` function can be used to display a concise overview of the data frame's internal structure, including the name, type, and a preview of the data in each column.

```r
str(dat)
```

```
## 'data.frame':    100 obs. of  9 variables:
##  $ ID           : chr  "Sub001" "Sub002" "Sub003" "Sub004" ...
##  $ Gender       : chr  "m" "m" "m" "f" ...
##  $ Group        : chr  "Control" "Treatment2" "Treatment2" "Treatment1" ...
##  $ BloodPressure: int  132 139 130 105 125 112 173 108 131 129 ...
##  $ Age          : num  16 17.2 19.5 15.7 19.9 14.3 17.7 19.8 19.4 18.8 ...
##  $ Aneurisms_q1 : int  114 148 196 199 188 260 135 216 117 188 ...
##  $ Aneurisms_q2 : int  140 209 251 140 120 266 98 238 215 144 ...
##  $ Aneurisms_q3 : int  202 248 122 233 222 320 154 279 181 192 ...
##  $ Aneurisms_q4 : int  237 248 177 220 228 294 245 251 272 185 ...
```

The `str()` function informs us that the data consists of 100 observations (rows) and 9 variables (columns). It also indicates that the data frame comprises character, integer, and numeric vectors (sets of values).

```r
head(dat)
```

```
##       ID Gender      Group BloodPressure  Age Aneurisms_q1 Aneurisms_q2
## 1 Sub001      m    Control           132 16.0          114          140
## 2 Sub002      m Treatment2           139 17.2          148          209
## 3 Sub003      m Treatment2           130 19.5          196          251
## 4 Sub004      f Treatment1           105 15.7          199          140
## 5 Sub005      m Treatment1           125 19.9          188          120
## 6 Sub006      M Treatment2           112 14.3          260          266
##   Aneurisms_q3 Aneurisms_q4
## 1          202          237
```

```
## 2          248          248
## 3          122          177
## 4          233          220
## 5          222          228
## 6          320          294
```

The `head()` function prints out the first 6 rows of the data. This data represents the results of a hypothetical experiment, examining the number of aneurysms that formed in patients' eyes following three different treatments.

Other useful data frame functions include:

- `tail()`: Prints out the last 6 rows.
- `dim()`: Shows the dimension of data frame – i.e., the number of rows and columns.
- `nrow()`: Shows the number of rows
- `ncol()`: Shows the number of columns

```r
tail(dat)
```

```
##         ID Gender      Group BloodPressure  Age Aneurisms_q1 Aneurisms_q2
## 95  Sub095      m    Control           108 13.6          111          118
## 96  Sub096      m    Control           102 14.6          148          132
## 97  Sub097      F Treatment2            90 19.6          141          196
## 98  Sub098      m Treatment1           133 17.0          193          112
## 99  Sub099      M Treatment2            83 16.2          130          226
## 100 Sub100      M Treatment1           122 18.4          126          157
##     Aneurisms_q3 Aneurisms_q4
## 95           173          191
## 96           200          194
## 97           322          273
## 98           123          181
## 99           286          281
## 100          129          160
```

```r
dim(dat)
```

```
## [1] 100   9
```

```r
nrow(dat)
```

```
## [1] 100
```

```r
ncol(dat)
```

```
## [1] 9
```

3.2. Referencing by Index

Data can be referenced by *index*. You can use square square brackets `[]` to subset data (sometimes also referred to as "slicing"). The generic format is `dat[row_numbers, column_numbers]`.

```r
dat[1, 1]
```

```
## [1] "Sub001"
```

```r
dat[5, 4]
```

```
## [1] 125
```

Finally, to reference *non-sequential* rows and columns, you can use the `c()` function. The following code returns the first five columns for patients in rows 1, 5, 7, and 9.

```r
dat[c(1, 5, 7, 9), 1:5]
```

```
##       ID Gender      Group BloodPressure  Age
## 1 Sub001      m    Control           132 16.0
## 5 Sub005      m Treatment1           125 19.9
## 7 Sub007      f    Control           173 17.7
## 9 Sub009      m Treatment2           131 19.4
```

[**Exercise**]

- Write a code to return the Gender and Age values for the first five patients.
- What will `dat[2,  ]` return?
- What will `dat[ , 3]` return?

3.3. Referencing by Name

Columns in an R data frame are named.

```r
names(dat)
```

```
## [1] "ID"            "Gender"        "Group"         "BloodPressure"
## [5] "Age"           "Aneurisms_q1"  "Aneurisms_q2"  "Aneurisms_q3"
## [9] "Aneurisms_q4"
```

[Note] If column names are not specified (e.g., `headers = FALSE` in the `read.csv()` function), R assigns default names V1, V2, . . . , Vn.

We typically use the `$` operator to refer to a column by its name.

```r
dat$Gender
```

```
##   [1] "m" "m" "m" "f" "m" "M" "f" "m" "m" "f" "m" "f" "f" "m" "m" "m" "f" "m"
##  [19] "m" "F" "f" "m" "f" "f" "m" "M" "M" "f" "m" "f" "f" "m" "m" "m" "m" "f"
##  [37] "f" "m" "M" "m" "f" "m" "m" "m" "f" "f" "M" "M" "m" "m" "m" "f" "f" "f"
##  [55] "m" "f" "m" "m" "m" "f" "f" "f" "f" "M" "f" "m" "f" "f" "M" "m" "m" "m"
##  [73] "F" "m" "m" "f" "M" "M" "M" "f" "m" "M" "M" "m" "m" "f" "f" "f" "m" "m"
##  [91] "f" "m" "F" "f" "m" "m" "F" "m" "M" "M"
```

Name referencing can also be used with square brackets `[]`.

```r
head(dat[ , c("Gender", "Age")])
```

```
##   Gender  Age
## 1      m 16.0
## 2      m 17.2
## 3      m 19.5
## 4      f 15.7
## 5      m 19.9
## 6      M 14.3
```

[Note] As a best practice, it's advisable to refer to columns by their name. The reason being, column positions can change as you add or delete columns.

4.4. Referencing by Logical Vector

A logical vector contains the special values `TRUE` and `FALSE`. Please note that the values `TRUE` and `FALSE` are in all capital letters and are not enclosed in quotes.

```r
c(TRUE, TRUE, FALSE)
```

```
## [1]  TRUE  TRUE FALSE
```

Logical vectors can be created using *relational operators* such as `<`, `>`, `==`, `!=`, `%in%`, etc.

```r
x <- c(1, 2, 3, 11, 12, 13)
x < 10
```

```
## [1]  TRUE  TRUE  TRUE FALSE FALSE FALSE
```

```r
x %in% 1:10
```

```
## [1]  TRUE  TRUE  TRUE FALSE FALSE FALSE
```

You can use logical vectors to select data from a data frame, an operation often called as *logical indexing*.

```r
ctr <- dat$Group == "Control"
ctr
```

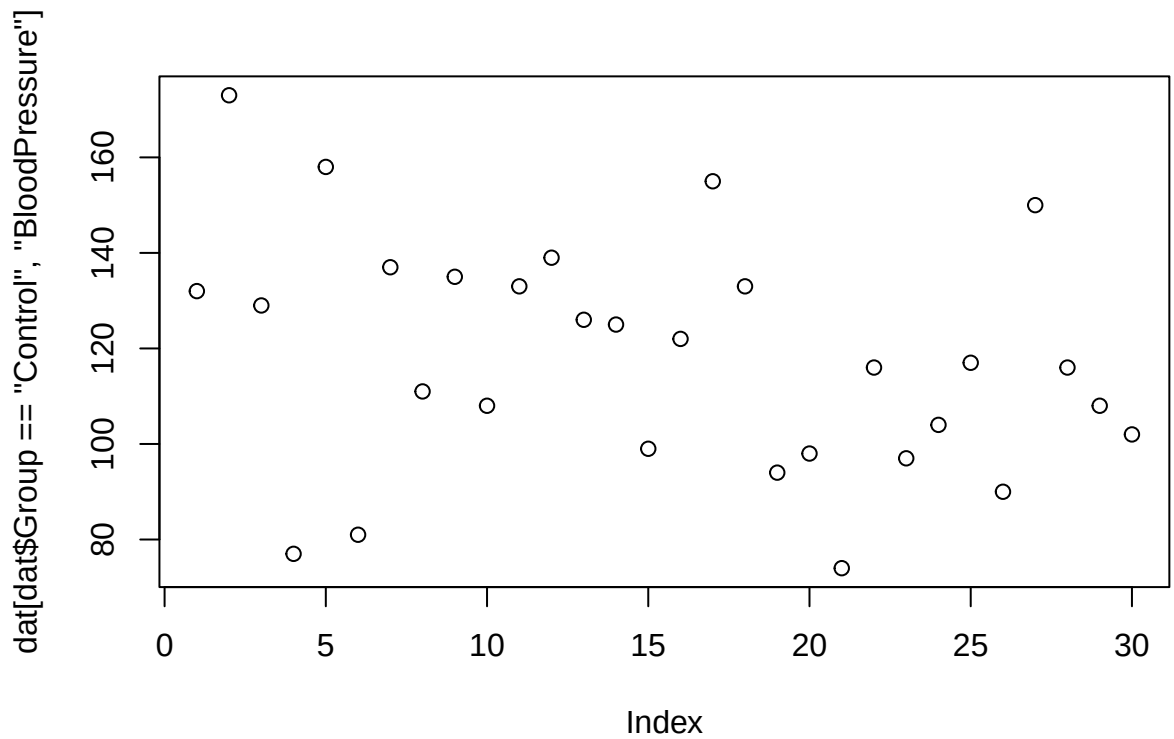
```
##   [1]  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE  TRUE FALSE FALSE
##  [13]  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE
##  [25] FALSE FALSE FALSE FALSE  TRUE  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE
##  [37] FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE  TRUE FALSE  TRUE  TRUE
##  [49]  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE  TRUE
##  [61]  TRUE FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE  TRUE FALSE  TRUE
##  [73] FALSE  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE
##  [85] FALSE  TRUE  TRUE FALSE FALSE  TRUE FALSE FALSE  TRUE FALSE  TRUE  TRUE
##  [97] FALSE FALSE FALSE FALSE
```

```r
dat[ctr, "BloodPressure"]
```

```
##  [1] 132 173 129  77 158  81 137 111 135 108 133 139 126 125  99 122 155 133  94
## [20]  98  74 116  97 104 117  90 150 116 108 102
```

Typically, this operation is written in a single line of code.

```r
plot(dat[dat$Group == "Control", "BloodPressure"])
```



3.5. Combining Referencing and Assignment

The assignment operator `<-` can be used in conjunction with referencing.

```r
x <- c(1, 2, 3, 11, 12, 13)
x[x < 10] <- 0
x
```

```
## [1]  0  0  0 11 12 13
```

[**Exercise**]

In this dataset, gender values have been recorded as both uppercase (M, F) and lowercase (m, f). Utilize the combination of referencing and assignment operations to convert all values to lowercase.

```r
dat[dat$Gender == "M", ]$Gender <- "m"
dat[dat$Gender == "F", ]$Gender <- "f"
```

4. Understanding Factors

Objectives

- Understand how categorical data is represented in R.
- Know the difference between ordered and unordered factors.
- Be aware of some common problems that arise when using factors.

In R, factors are employed to represent categorical data. These factors can be either ordered or unordered, and they play a vital role in statistical analysis and plotting.

While factors may appear to be (and often behave like) character vectors, they are actually integers at their core. Therefore, caution is needed when treating them as strings.

4.1. `factor()` Function

The `factor()` function is used to create and modify factors.

Once factors are created, they can only contain a predefined set values, known as *levels*. By default, R sorts these levels in *alphabetical order*. For instance, if you have a factor with two levels:

```r
sex <- factor(c("male", "female", "female", "female"))
```

R will assign 1 to the level "female" and 2 to the level "male" (because "f" precedes "m" alphabetically, even though the first element in this vector is "male"). You can check this using the `levels()` function, and check the number of levels using `nlevels()`.

```r
levels(sex)
```

```
## [1] "female" "male"
```

```r
nlevels(sex)
```

```
## [1] 2
```

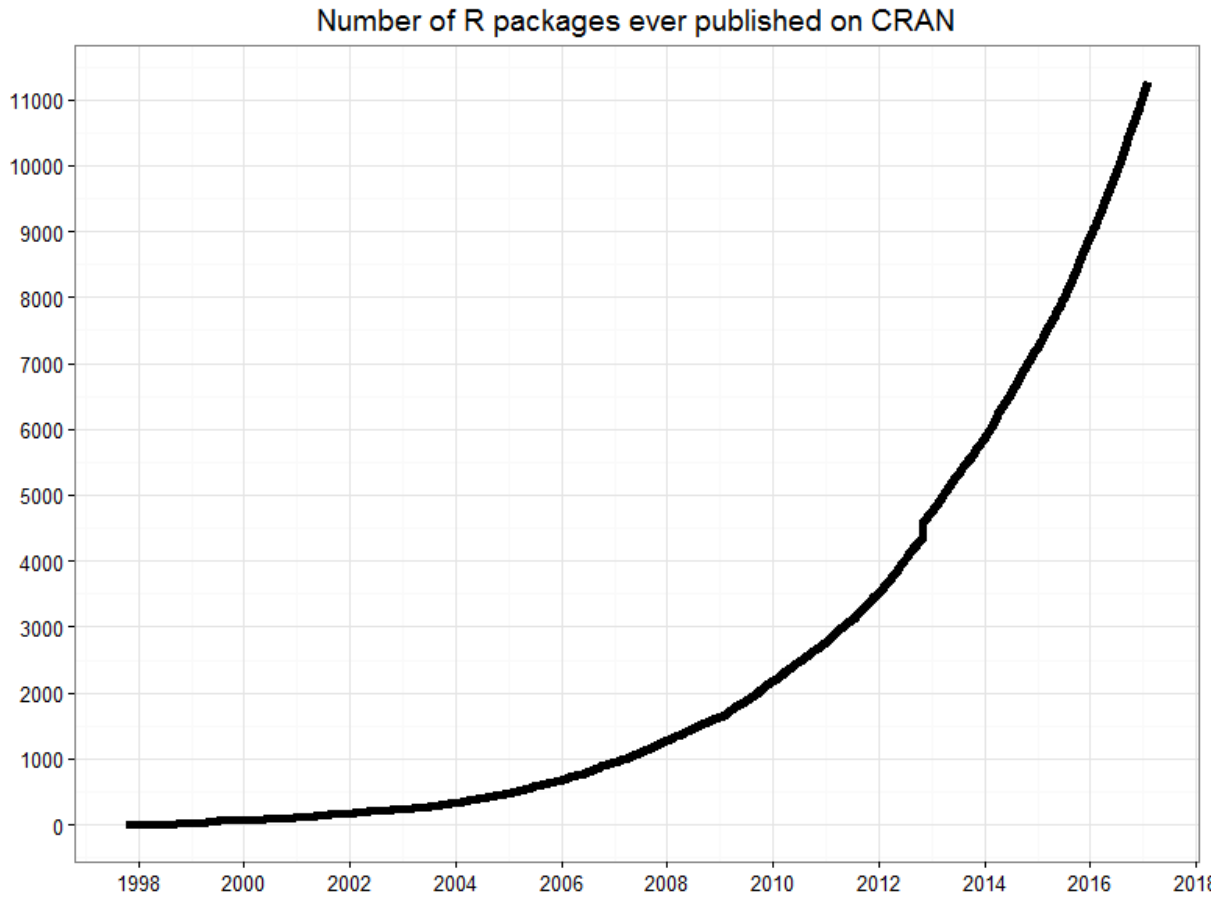
5. Packages & Reading and Writing Data Files

Objectives

- Understand packages in R.
- Learn how to install and load packages.
- Learn the basics of the `tidyverse()` Package.
- Learn to import different types of data files into R (csv, xls, sav).
- Understand how to write data files.

5.1. Packages in R

Functions such as `read.csv()`, `summary()`, and `factor()` come built-in with R. However, sometimes we require functions beyond those provided in base R to meet our goals (for tasks such as data importing, cleaning, analysis, and visualization). For these purposes, we'll need to install various packages. Think of packages as toolboxes, with each function within a package being a tool. The Comprehensive R Archive Network (CRAN) houses over 20,000 such "toolboxes".



- To manage the packages already installed on your system, click the **Packages** tab in the bottom right panel of RStudio.
- To load a package, check the box next to the package name. Alternatively, you can use the `library()` function from the Script Editor.
- To install a package, click the **Install** icon » type in the package name » **Install**. Alternatively, you can use the `install.packages()` function from the Script Editor.

Let's proceed by installing a package named `tidyverse`.

tidyverse is a comprehensive collection of R packages specifically designed for data science. All packages within it share a common design philosophy, grammar, and data structures.

```
### Install a package.
# install.packages("tidyverse")
```

```r
### Load the package
library(tidyverse)
```

[Note]

- Packages only need to be installed once.
- You must load the packages each time you open a **new R session**.
- For detailed informatoin on a package, refer to the package manual (e.g., search for the package name and CRAN on Google), or use `?` followed by the package name.
- For guidance on specific functions, refer to the package manual, use `?` followed by the function name, or search on R Documentation.
- Stack Overflow is another great resource.

5.2. Read and Write Data Files

There are various formats of data files, depending on the nature of the data (txt, csv, delimited, html, etc). Moreover, you may come across different types of datasets depending on the statistical software used (SAS, SPSS, STATA, Mplus, Minitab, etc). We will focus on a few of them:

- **csv** (plain text format with comma-separated values)
- **xls/xlsx** (Excel)
- **sav** (SPSS)

5.2.1. Read csv File

Earlier in the class, we imported a dataset using the `read.csv()` function:

```
dat <- read.csv(file = "sample1.csv", header = TRUE)
```

Now, we'll attempt to read a csv file into R using `tidyverse`. The function we will use is `read_csv()`. The `file = 'sample.csv'` argument works in the same way as in the `read.csv()` function, and the `col_names = TRUE` argument just works like `header = TRUE`.

```r
library(tidyverse)

dat1 <- read_csv(file = "sample1.csv", col_names = TRUE)
```

5.2.2. Read xls/xlsx File

```r
library(readxl)

dat2 <- read_excel(path = "sample2.xlsx", sheet = 1, col_names = TRUE)
```

5.2.3. Read sav File

```r
library(haven)

dat3 <- read_sav(file = "sample3.sav")
```

```r
write_csv(x = dat2, file = "output1.csv", na = "NA")

# write_csv(dat, "output1.csv", na = "-999")

### Also in base R
# write.csv(dat3, "output1.csv")
```

5.2.4. Write csv File *[Note]*

- Ensure that the data files are stored in the same working directory that you have set for the R session.
- The file that you write will be saved into the working directory.
- There are usually more than one way to import data. For instance, the `foreign` package can also handle SPSS, SAS, Stata, and many other file types.
- csv is one of the most commonly adopted formats.

5.3. `tydyverse` Package

When working with `tidyverse`, we can use the `%>%` operator (called 'pipe') to link our operations with less typing (we could read it as 'then' for now). Use `Ctrl + Shift + M` shortcut (`Cmd + Shift + M` for macOS). We will take `dat1` as an example, to perform some simple data manipulations.

5.3.1. `select()` for extracting variables Let's try:

1. Select demographic variables by column index.
2. Select variables by specific column name.
3. Select variables by column naming scheme/pattern.
4. Select order will determine the order of new data frame.
5. Drop variables by adding a minus sign in front.

```r
dat_demogr <- select(dat1, c(1:3, 5))

dat_exmp <- select(dat1, BloodPressure, Aneurisms_q1, Aneurisms_q2)

dat_Aneu <- select(dat1, starts_with("Aneurisms_"), ends_with("Pressure"))

dat_order <- select(dat1, c(1,3, 5, 2, 6:9))

dat_drop <- select(dat1, -c(1:3, 5))
```

5.3.2. `mutate()` / `mutate_if()` for manipulating variables Let's try:

1. Convert **ID**, **Gender**, and **Group** from `chr` to `fctr` using `mutate_if()`.
2. Create a new variable **Aneurisms_total** using `mutate` for calculation.
3. Re-code certain values within a variable.
4. Integrate the `%>%` operator.

```r
mydat <- dat1

mydat <- mutate_if(mydat, is.character, factor)

mydat <- mutate(mydat, Aneurisms_total = Aneurisms_q1 + Aneurisms_q2 + Aneurisms_q3 + Aneurisms_q4)

mydat <- mutate(mydat, Gender_re = recode(Gender, "M"="m", "F"="f"))

### To much typing? Use pipe!
mydat <- dat1 %>%
  mutate_if(is.character, factor) %>%
  mutate(Aneurisms_total = Aneurisms_q1 + Aneurisms_q2 + Aneurisms_q3 + Aneurisms_q4,
         Gender_re = recode(Gender, "M"="m", "F"="f"))
```

5.3.3. `filter()` for selecting cases The `filter()` function returns a subset of **rows** that meet logical criteria as a new data frame.

Let's try:

1. Find males who are younger than 18.
2. Find females who are not in the Control group (either in the Treatment1 or Treatment2 group).

```r
mydat_m <- mydat %>%
  filter(Gender_re == "m",
         Age <= 18)

mydat_f <- mydat %>%
  filter(Gender_re == "f",
         Group != "Control")

# Try this:
mydat_f2 <- mydat %>%
  filter(Gender_re == "f",
         Group == "Treatment1" | Group == "Treatment2")
```

5.3.4. `summarise()` (or `summarize()`) for producing summary statistics The `summarise()` function creates a new data frame. It will have one (or more) rows for each combination of grouping variables; if there are no grouping variables, the output will contain a single row summarizing all input observations. It will contain one column for each grouping variable and one column for each of the summary statistics you have specified.

Let's try:

1. Find the sample size, average age, and maximum blood pressure for different Gender groups.
2. Write the summary statistics into a csv file.

```r
lastquest <- mydat %>%
  group_by(Gender_re) %>%
  summarise(sample_size = n(), mean_age = mean(Age), max_bp = max(BloodPressure))

write_csv(lastquest, "yeah.csv")
```

6. Ploting in R (optional)

Objectives

- Understand the arguments used for generating plots in R.
- Learn and apply useful ploting functions.

6.1. Basic Ploting

In R, the generic function for creating plots is `plot()`. This function can be used to generate a variety of plot types depending on the data and arguments provided.

Common Arguments:

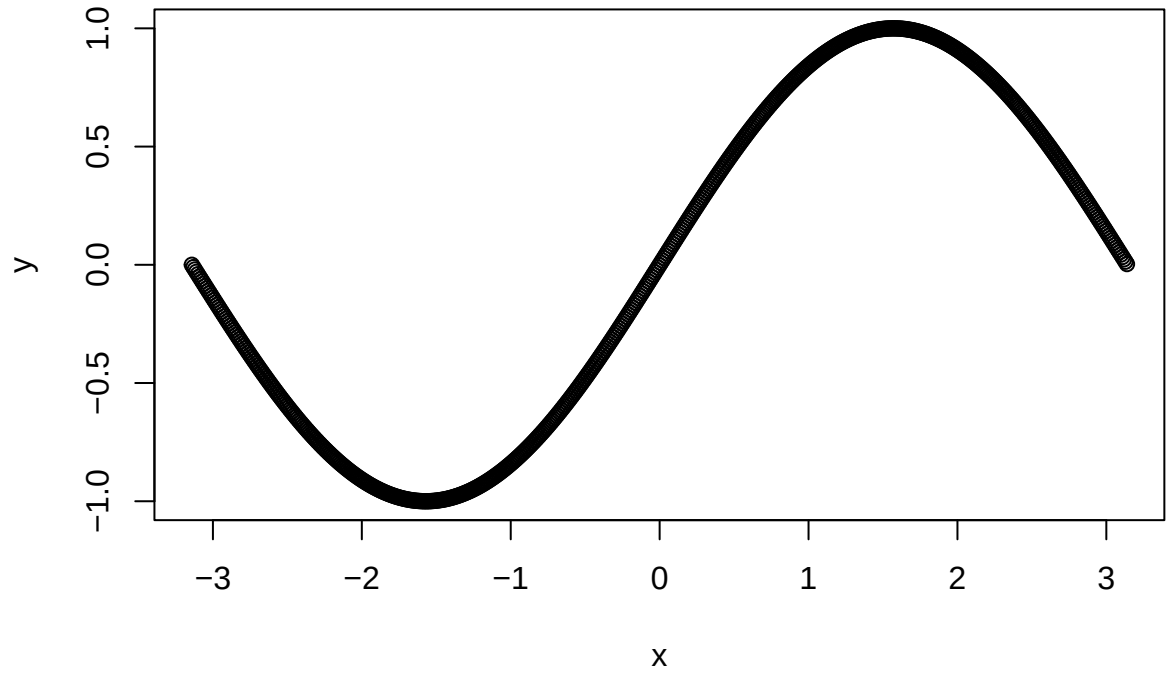
- **type**: Specifies the type of plot to create. Some common options include:
  - **p**: Points (scatter plot)
  - **l**: Lines
  - **b**: Both points and lines
  - **o**: Overplotted points and lines
  - **h**: Vertical lines (like a histogram)
- **main**: Defines the title of the plot, displayed at the top.
- **xlab**: Label for the x-axis.
- **ylab**: Label for the y-axis.

Additional Arguments:

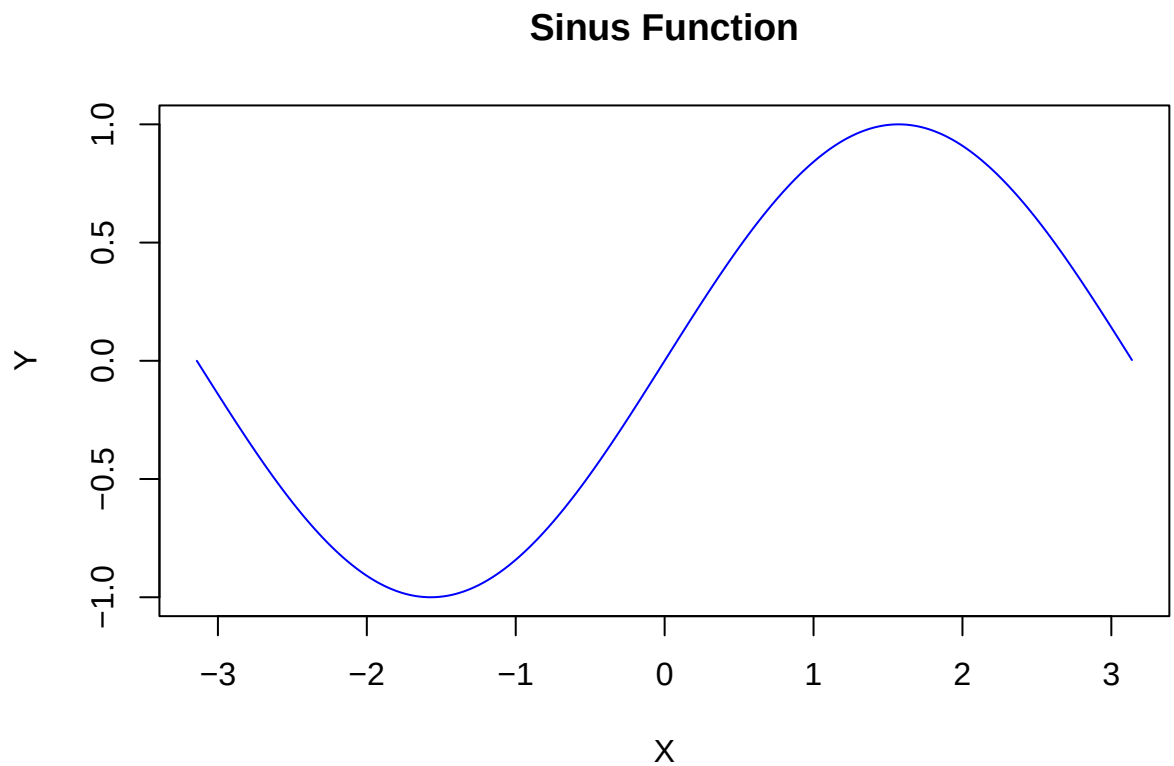
- **col**: Sets the color of the plot elements (points, lines, etc.).
- **pch**: Specifies the symbol (shape) used for points. For example, 1 for circles, 2 for triangles.
- **xlim**, **ylim**: Sets the limits for the x and y axes, respectively.
- **lwd**: Adjusts the line width for line plots.
- **cex**: Controls the size of points or symbols on the plot.

Examples:

```r
x <- seq(-pi, pi, .01)
y <- sin(x)
plot(x, y)
```




```r
plot(x, y, type = "l", main = "Sinus Function", xlab = "X", ylab = "Y", col = "blue")
```



6.2. Scatter Plot

A scatter plot displays values for typically two variables for a set of data. Each point on the plot represents one observation, with its position determined by the values of the two variables.

Scatter plots are useful for:

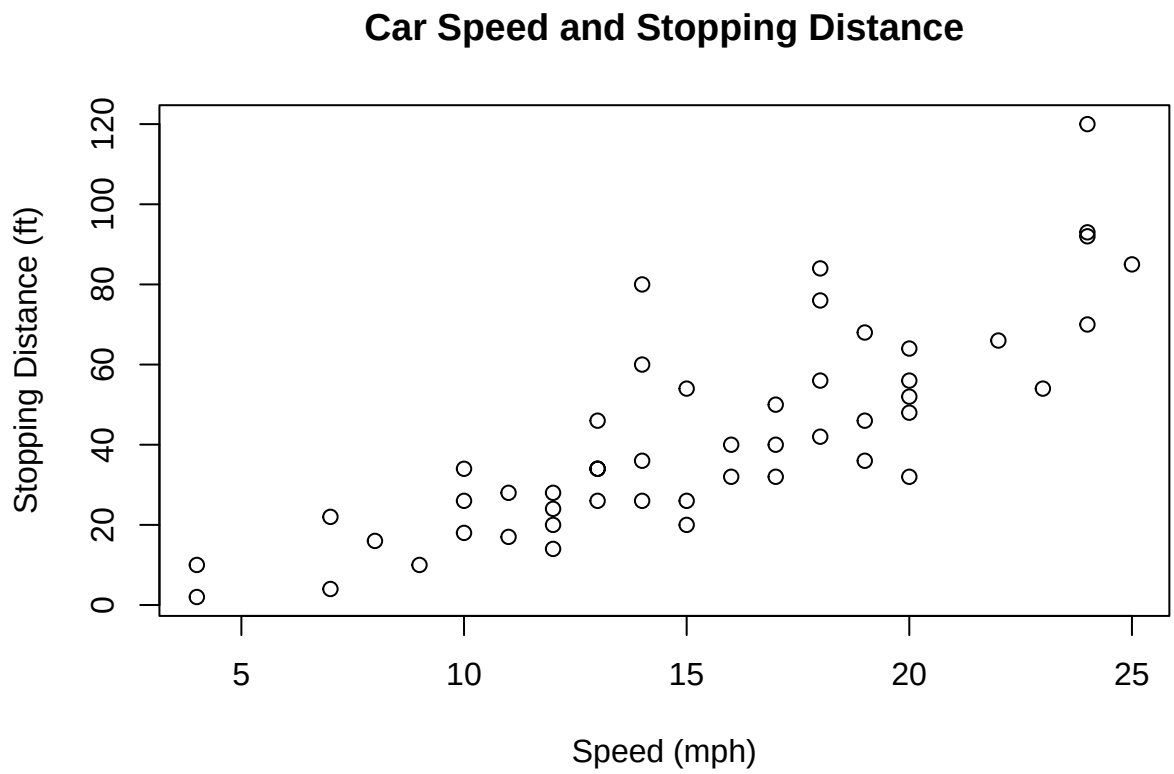
- Identifying relationships or correlations between variables
- Spotting trends, clusters, or outliers in the data
- Visualizing the spread and variability of data points

For example, if you wanted to see if there is a relationship between car speed (mph) and distance to stop (ft), a scatter plot would show how these two variables interact, with each point representing an experiment result.

```
head(cars)
```

```
##   speed dist
## 1     4    2
## 2     4   10
## 3     7    4
## 4     7   22
## 5     8   16
## 6     9   10
```

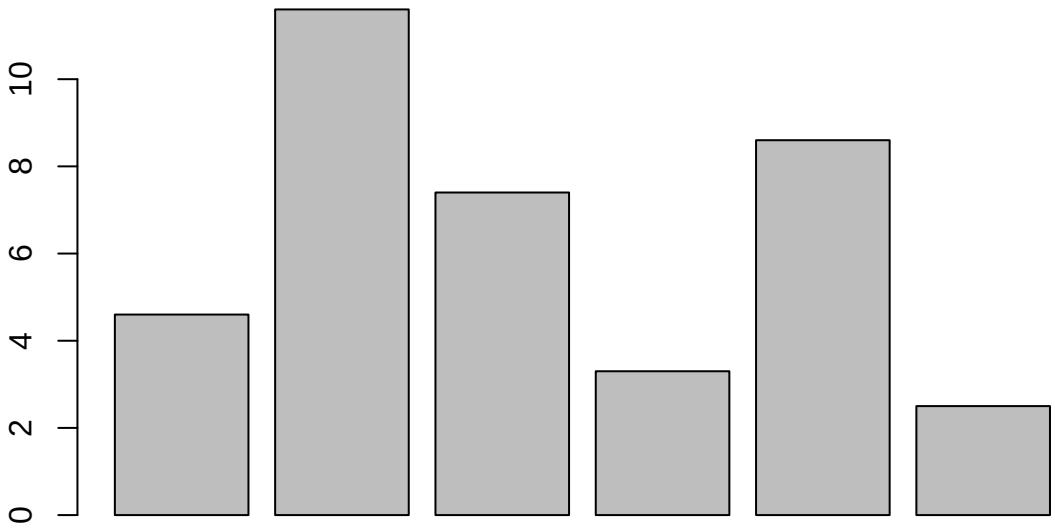
```r
plot(cars$speed, cars$dist, main = "Car Speed and Stopping Distance", xlab = "Speed (mph)", ylab = "Sto
```



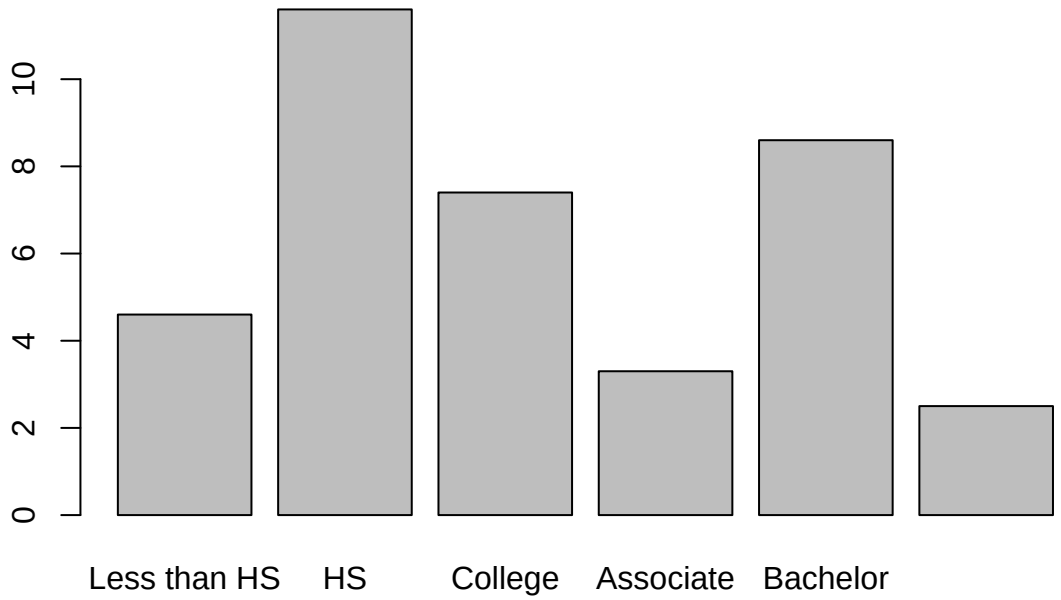
6.3. Bar Plot

Example:

```r
education <- data.frame(
  Level = c("Less than HS", "HS", "College", "Associate", "Bachelor", "Advanced"),
  Count = c(4.6, 11.6, 7.4, 3.3, 8.6, 2.5),
  Percent = c(12.1, 30.5, 19.5, 8.7, 22.6, 6.6)
)
barplot(education$Count)
```



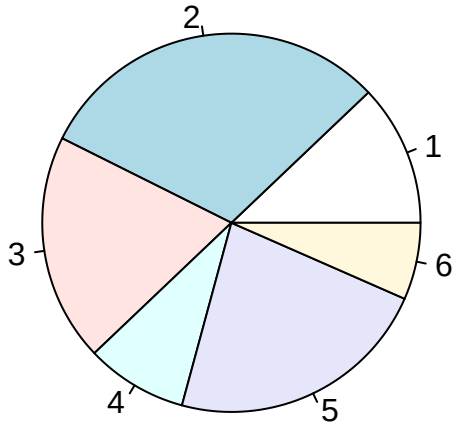

```r
barplot(education$Count, names.arg = education$Level)
```



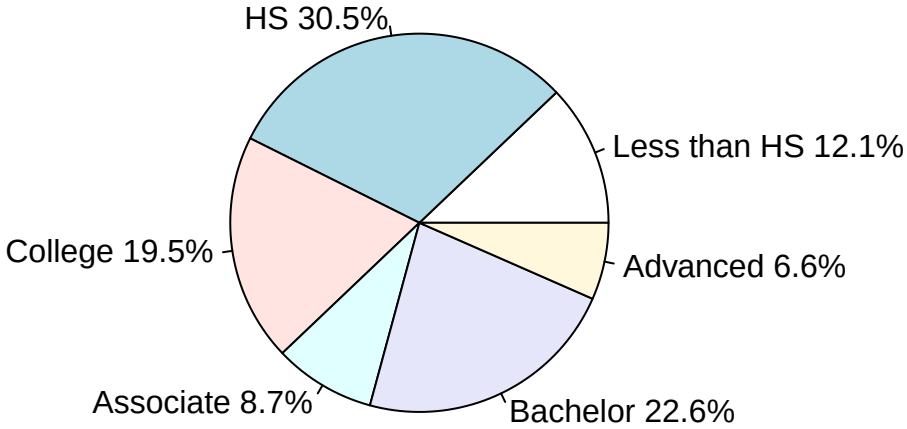
6.4. Pie Plot

Example:

```
pie(education$Count)
```



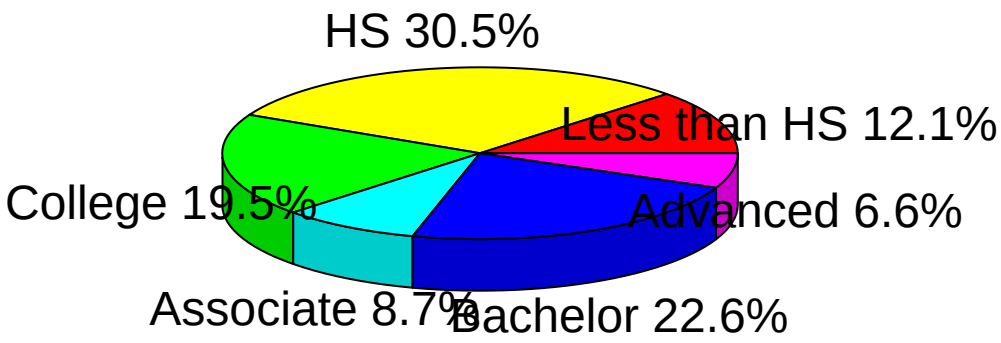

```
pielabels <- paste(education$Level, education$Percent)
pielabels <- paste(pielabels, "%", sep = "")
pie(education$Count, labels = pielabels)
```



6.5. 3D Pie Plot

Example:

```
# install.packages("plotrix")
library(plotrix)
pie3D(education$Count, labels = pielabels)
```

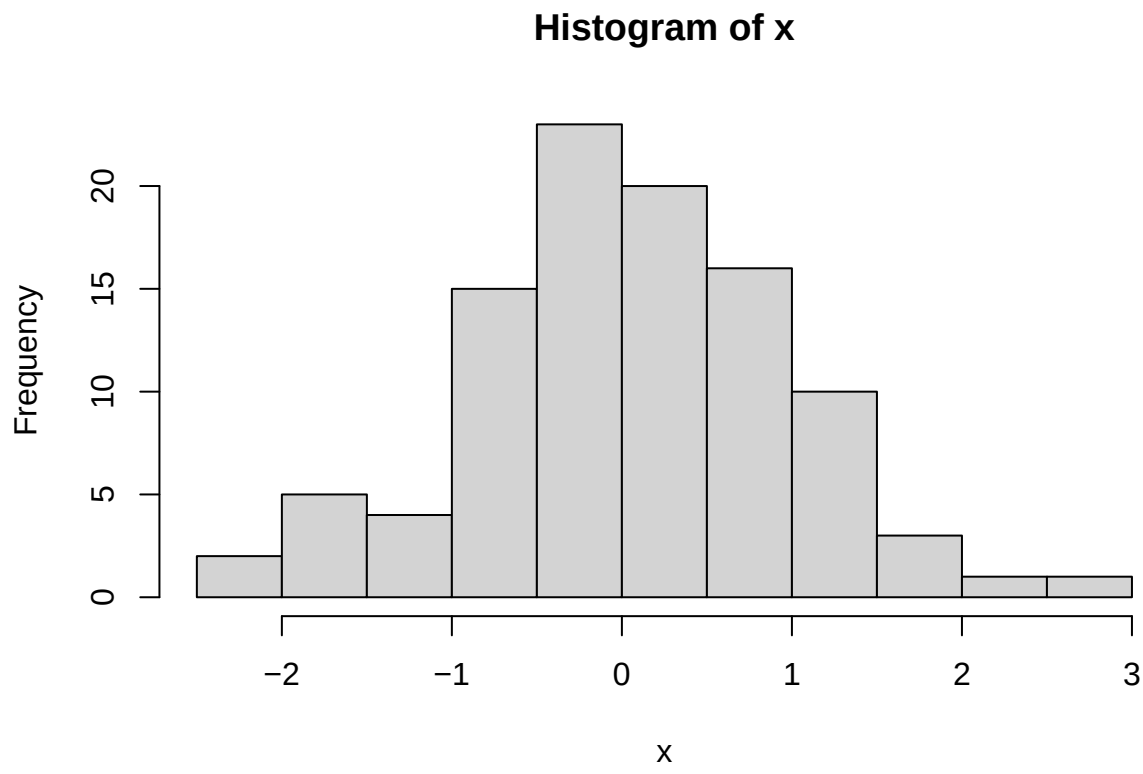


6.6. Histogram

A histogram is a type of bar chart that represents the distribution of numerical data. It divides data into bins (or intervals) and displays the frequency (count) of data points within each bin. The x-axis represents the range of values (or bins), and the y-axis shows the frequency of data points in each bin.

For example, suppose we have 100 random numbers generated by the `rnorm()` function.

```
x <- rnorm(100)
hist(x)
```



6.7. Box Plot

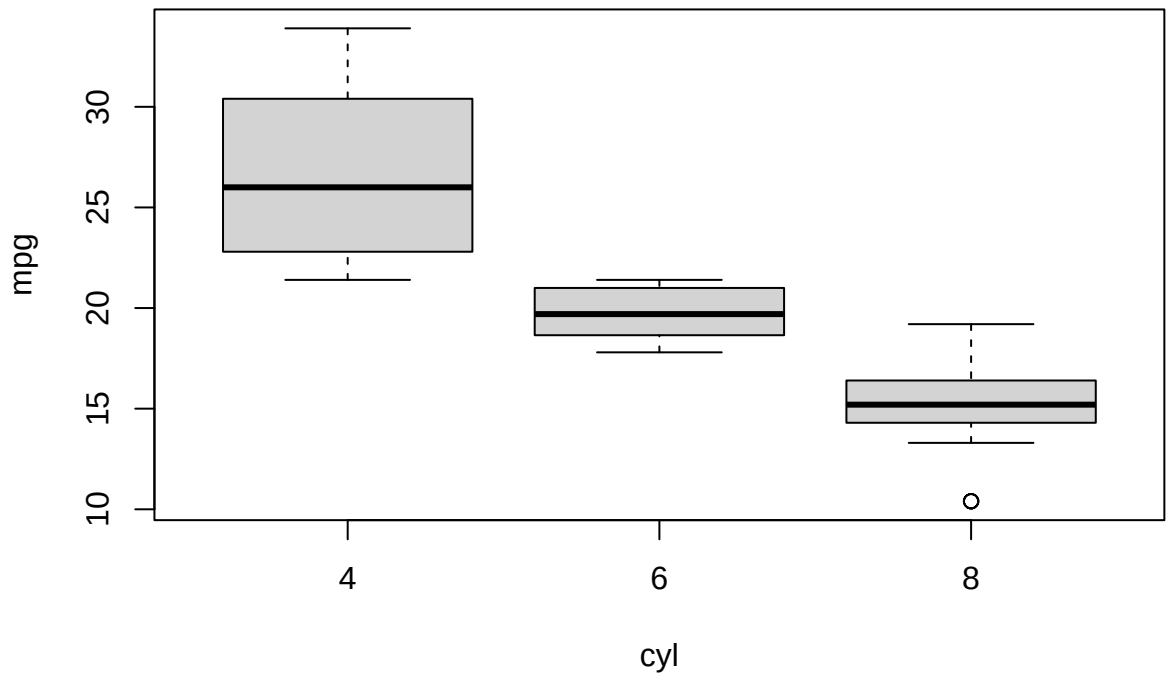
A box plot, also known as a **box-and-whisker** plot, is a standardized way of displaying the distribution of data based on a five-number summary: minimum, first quartile (Q1), median (Q2), third quartile (Q3), and maximum. It provides a visual summary that helps you understand the spread and skewness of the data, as well as identify outliers.

In this example, we are using the built-in data frame `mtcars` in R to create a box plot. The `attach()` function is used to make the variables in the data frame available by their names directly, so we don't need to use the mtcars$ notation each time we reference a variable.

```
library(ggplot2)
attach(mtcars)
```

```
## The following object is masked from package:ggplot2:
##
##     mpg
```

```
boxplot(mpg~cyl)
```


mpg
10
15
20
25
30
4
6
8
cyl